



北京理工大學珠海學院

BEIJING INSTITUTE OF TECHNOLOGY, ZHUHAI

基于机器学习的大学 生心理素质的分析与预测

学 院： 数理与土木工程学院

课 程： 数据挖掘

班 级： 数据科学与大数据技术 2 班

姓 名： 肖宇涵 学 号： 211205101786

中国·珠海

2023 年 12 月 27 日

基于机器学习的大学生心理素质的分析与预测

摘 要

随着各行各业互联网化的发展，大数据已经被广泛运用到了各行各业中。其中，心理学领域也开始应用大数据技术，利用各种计算机技术和数据分析手段来评估人的心理健康问题。本文就是对大学生心理素质的分析与预测，以达到提高大学生的心理健康水平的目标。本文从 CSDN 博客上获取数据并对其数据进行了清洗和预处理，使用了可视化的手段对数据进行了分析并使用了随机森林、决策树和 SVM 模型进行了是否会出现心理问题的预测。结果表明决策树在 Precision、Recall 和 F1-score 三个指标上较其他两个模型都有着更好的准确度。

关键词：心理素质 决策树 随机森林 SVM

目 录

1	引言	1
1.1	研究背景	1
1.2	意义	1
1.3	研究现状	2
2	数据描述	3
2.1	数据集	3
2.2	数据分布描述统计信息	4
2.3	数据预处理	4
3	数据分析	5
3.1	各年级人数分布	5
3.2	性别分布	5
3.3	参与人数排名前 10 的学院	6
3.4	强迫症等级分布	6
3.5	阳性项目均分分布	7
3.6	重要特征之间的散点图	7
3.7	各特征相关系数	8
4	模型和算法简介	8
4.1	决策树 (Decision Tree)	8
4.1.1	原理	8
4.1.2	关键概念	9
4.1.3	构建过程	9
4.2	支持向量机 (SVM)	9
4.2.1	原理	9
4.2.2	关键概念	10
4.2.3	优化问题	10
4.2.4	核函数 (Kernel Function)	10
4.3	随机森林 (Random Forest)	10
4.3.1	原理	10
4.3.2	构建过程	11
4.4	混淆矩阵 (Confusion Matrix)	11
4.4.1	定义	11

4.4.2	组成	12
4.4.3	主要指标	12
4.5	ROC 曲线 (Receiver Operating Characteristic Curve)	12
4.5.1	定义	12
4.5.2	关键概念	13
4.5.3	AUC (Area Under Curve)	13
5	模型结果分析	13
5.1	决策树	13
5.2	SVM	14
5.3	随机森林	15
5.4	综合比较	16
5.5	考虑因素	16
6	总结与评价	16
6.1	总结	16
6.2	评价	17
7	模型改进建议	17
	附录	20

1 引言

1.1 研究背景

李金鹏^[1] 大学生是年轻人成长发展的重要阶段，他们具有高度的可塑性和活力。大学生的心理健康问题是各个国家都普遍关心的问题，他们的教育水平和智力水平与国家的发展有着密切的关系。大学生的心理素质不仅关系到个人的身心健康，还直接影响到他们的学业成绩、社会交往、就业等方面的发展。因此，研究大学生的心理素质具有重要意义。当前，互联网和信息技术的快速发展和普及促进了各行各业的数字化转型，因此数字化技术和大数据应用变得越来越重要。大数据是指海量、多样、高速的数据处理技术，可以获取大量数据、处理分析并发掘数据中的价值。面对大量的数据，研究者可以利用这些数据进行心理素质分析。基于大数据的大学生心理素质分析与预测的研究需要有以下背景：

1. 大学生心理健康问题突出。随着大学生学习和生活压力的增大，大学生心理健康问题越来越成为关注的焦点，包括抑郁、焦虑、人际交往困难等问题。
2. 数字化技术和大数据应用的兴起。数字化技术和大数据应用的快速发展，为心理素质分析提供了强有力的支持和技术手段。
3. 提高大学生心理素质的需求。近些年，心理健康教育在大学生中越来越受重视。通过研究大学生的心理素质，可以更好地为大学生提供有针对性的心理健康教育和指导。

研究内容包括：

1. 数据采集。收集大量的大学生心理素质相关数据，包括个体心理素质评估、心理测试结果、以及其他行为数据等。
2. 数据整合和分析。将收集到的数据整合并加以分析，通过统计方法和机器学习算法挖掘其中的规律和特征，从而提取针对大学生心理素质评估的关键因素。
3. 模型建立与预测。通过建立模型，将关键因素和心理素质结果相联系，从而进行心理素质预测。利用算法可以更准确地预测大学生的心理素质发展，指导和优化大学生心理健康教育。

1.2 意义

钟晓虹等人^[2] 大数据在当今社会扮演着越来越重要的角色。它可以在大学生心理素质方面提供极大的研究价值。利用大数据分析出大学生典型心理问题、心理素质特点，为心理专业的科学研究提供大量的一线数据。

首先，基于大数据的大学生心理素质分析和预测对于提高大学生心理健康水平非常重要。心理障碍已成为影响当代大学生健康的一大问题。通过大数据分析和预测，我

们可以获取大型样本统计数据,分析大学生心理健康现状,洞察存在的问题,并预测心理健康风险。这将为学生个体心理干预提供更准确的建议和指导,帮助更多的学生提高心理健康水平。

其次,基于大数据的大学生心理素质分析和预测还可以优化教育教学。大数据分析工具可以对教学中的数据进行多角度和多维度的分析,挖掘学生的学习风格和特点,为教师制定更贴近学生实际和有效的教学方案提供数据支持。同时,预测学生成绩和学习需求,为优化师生关系和提高教育效率提供参考。

最后,基于大数据的大学生心理素质分析和预测还可以适应社会需求。随着社会的快速发展,社会需求在不断变化。秦娟^[3]心理教育是一种以心理学的原理与方法技术为主要依托并结合学校日常教学工作,有目的、有计划地培养学生良好心理素质的教育。基于大数据的大学生心理素质分析和预测可以更加准确地把握社会需求和趋势,为学生提供更恰当和适应社会的教育和培训,帮助更多学生迎接未来竞争和挑战。

总之,基于大数据的大学生心理素质分析和预测具有重要的现实和理论意义,将大大促进大学生心理健康、教育教学甚至社会需求的发展。

1.3 研究现状

目前,国内在心理素质和情感分析方面的研究还处于起步阶段,但是在一些前沿研究中,大数据技术已被应用。例如,一些研究学者利用大数据技术,从大量的文本数据中提取出心态状态和情感,从而探究心理状态。此外,一些研究还利用大数据技术来对大学生的人格特征进行分析和预测。

岳扬等人^[4]采用一种结合关键词提取和机器学习的情感分析方法对东航 MU5735 坠毁事件相关的部分微博数据进行情感分析,统计、分析坠机事件发生后的微博热搜话题数据。研究发现,公众对该事件以中性情感为主,消极情感次之。本文还结合传播学、心理学、社会学、统计学多角度对舆情进行解读。

完颜兵等人^[5]通过构建词云图进行热词分析;通过 DBSCAN 密度聚类、构建语义网络关系图完成特色分析;通过提出一种基于 Bert 的双路神经网络融合的文本情感分类模型 TNNFMB 结合迁移学习实现情感分析,以此挖掘游客的潜在印象。

彭梅等人^[6]通过利用大数据爬虫技术在电商网店中爬取某个商品的好评或差评大数据后,采用基于决策树情感分析、情感词库分析、LDA 模型的主题分析,综合 3 种方法对评论数据进行处理,得到一份最终的情感分析,从而得到消费者偏好的产品方向,以便商家能够获取进一步提升的方法。

张黎娜等人^[7]利用 Python 中的 jieba 分词对成人学习者在线聊天记录进行中文分词,并生成词频表的方法,同时通过朴素贝叶斯分类器进行文本情感倾向分析,并生成正面、负面、中立三个情感文本文档;最后利用 LDA 主题模型对情感文本进行主题聚类,得到不同情感类别的主题情感词。

房立艳等人^[8]通过 Logistic 回归分析结果显示,与“低拖延组”相比,在其余两组中,男生人数较多,初中生人数较多,性别和学段效应显著。同时,心理素质对拖延的影响显著,低心理素质的学生在“高拖延组”和“混合组”中人数较多。

金琳等人^[9]通过多层线性分析发现师生关系促进了父亲积极教养方式对中学生心理素质的正向影响,而学习负担和同学关系缓解了父亲消极教养方式对中学生的负向影响。

陈小红等人^[10]使用可视化分析大学生心理健康的相关研究以教育为主,研究内容丰富多样,但教育形式较少,互联网线上教育的形式或有新的探索意义。

综上所述,国内在心理素质和情感分析方面的研究还处于起步阶段,但大数据技术已经开始应用于这一领域。一些研究学者利用大数据技术从大量的文本数据中提取出心态状态和情感,以探究心理状态。另外,一些研究也利用大数据技术对大学生的人格特征进行分析和预测。

2 数据描述

2.1 数据集

本研究使用的数据集(见表1)来源于大一新生心理测试数据集,共11794行数据,18列字段,其中结果状态为0的是心理不太好的,结果状态为1的是心理正常的

表1: 字段说明

字段名	说明	非空值数量	数据类型
年级	学生的年级	11794	int64
性别	学生的性别	11794	object
学院	学生所在的学院	11794	object
总分	学生的总分数	11794	int64
躯体化均分	躯体化问题的平均分	11794	float64
强迫症均分	强迫症状的平均分	11794	float64
人际关系均分	人际关系问题的平均分	11794	float64
抑郁均分	抑郁症状的平均分	11794	float64
焦虑均分	焦虑症状的平均分	11794	float64
敌对均分	敌对行为的平均分	11794	float64
恐怖均分	恐怖感受的平均分	11794	float64
偏执均分	偏执症状的平均分	11794	float64
精神病性均分	精神病性症状的平均分	11794	float64
其他均分	其他问题的平均分	11794	float64
总均分	所有问题的总平均分	11794	float64
阳性项目均分	阳性反应项目的平均分	11794	float64
阳性项目数	阳性反应项目的总数	11794	int64
结果状态	表示结果的状态,二分类变量	11794	int64

随后进行部分重要数据的展示，见表 2

表 2: 重要数据数据展示

年级	性别	总分	...	人际关系均分	抑郁均分	焦虑均分	...	结果状态
18	男	158	...	2.44	1.77	1.5	...	1
18	男	239	...	3.33	2.85	2.4	...	0
18	女	115	...	1.33	1.15	1.3	...	1
18	女	115	...	1.56	1.15	1.4	...	1
18	女	245	...	3.00	2.92	2.8	...	0
...	...	...	...	...	...	...	...	...
20	女	107	...	1.00	1.08	1.1	...	1
20	女	98	...	1.11	1.15	1.1	...	1
20	女	315	...	3.78	3.69	4.1	...	0
20	女	167	...	2.56	1.85	1.9	...	0
20	男	172	...	2.67	1.77	1.6	...	0

2.2 数据分布描述统计信息

表 3: Descriptive Statistics of the Dataset

Statistic	年级	总分	躯体化均分	强迫症均分	...	阳性项目数	结果状态
Count	11794	11794	11794	11794	...	11794	11794
Mean	19.09	133.14	1.30	1.80	...	30.40	0.72
Std Dev	0.82	38.42	0.38	0.58	...	21.11	0.45
Min	18.00	90.00	1.00	1.00	...	0.00	0.00
25%	18.00	105.00	1.00	1.40	...	14.00	0.00
50%	19.00	123.00	1.17	1.70	...	27.00	1.00
75%	20.00	150.00	1.42	2.10	...	44.00	1.00
Max	20.00	418.00	5.00	5.00	...	90.00	1.00

2.3 数据预处理

查看数据集，发现没有缺失值。

发现重复值：有 412 条重复记录，数据集共 11794 行数据，影响不大，故删去

3 数据分析

3.1 各年级人数分布

从下图中可以看出在参与心理素质调查的人数中 20 级的人最多有 4428 人，其次是 19 级的有 3652 人，18 级参与调查的有 3302 人。

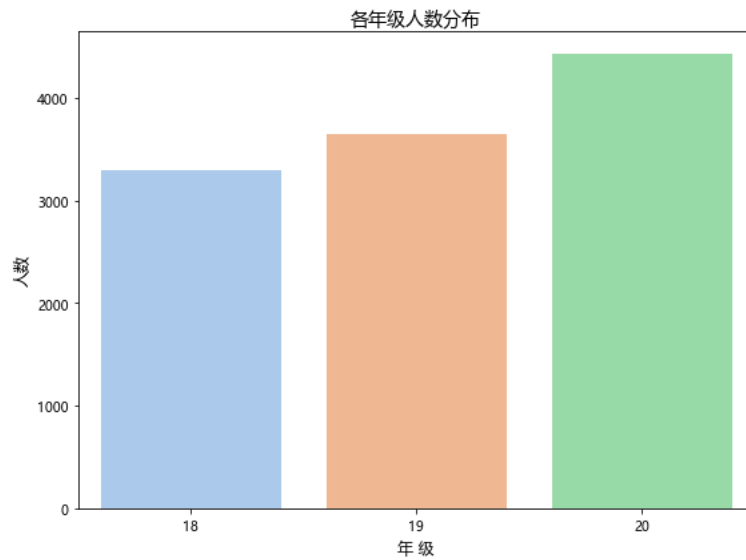


图 1: 各年级人数分布

3.2 性别分布

从下图中可以看出，参与心理素质检测的学生中女生占 71.4

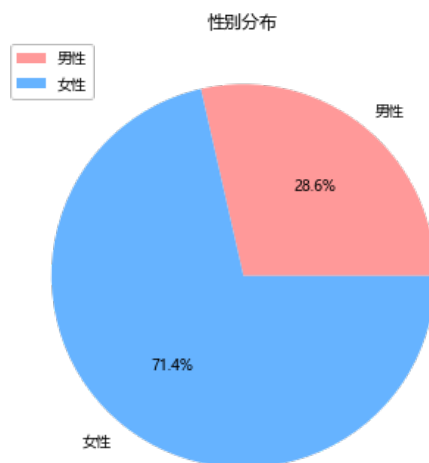


图 2: 性别分布

3.3 参与人数排名前 10 的学院

从下图中可以看出，其他学院的人数最多，其次是翰林学院，第三的是研究生院。

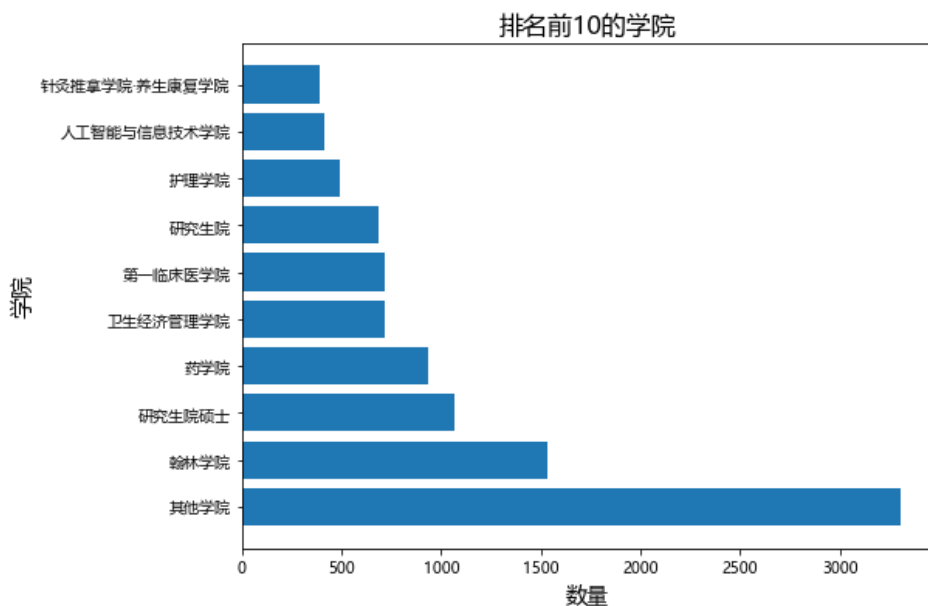


图 3: 参与人数排名前 10 的学院

3.4 强迫症等级分布

将强迫症等级按 0-1.5, 1.5-2.5 和大于 2.5 分为，正常、轻度和重度，从下图中可以看出在参与测试的学生中有轻度强迫症的学生最多，其次是正常的最后是重度强迫症的。从图中还可以看出，正常的人数加上重度的人数都还没有轻度的人多。

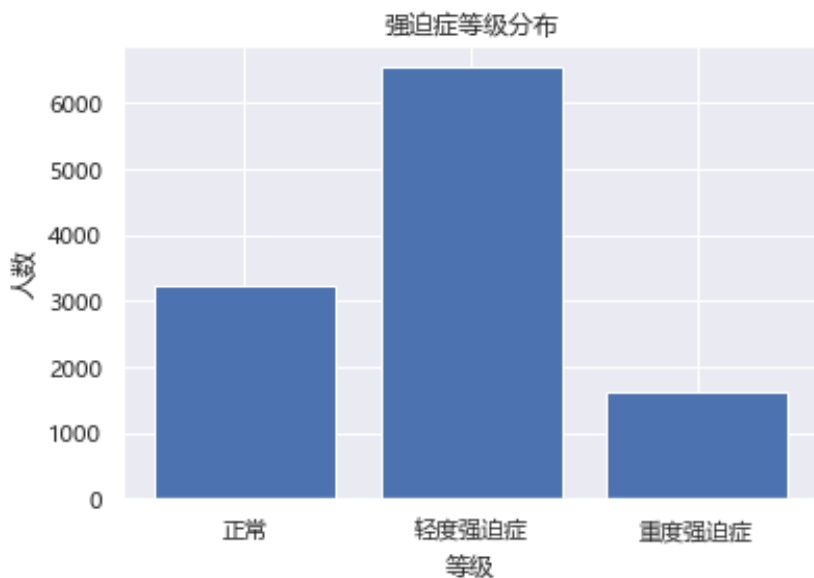


图 4: 强迫症等级分布

3.5 阳性项目均分分布

从下图中可以看出，阳性项目均分主要集中在 2-3 分。

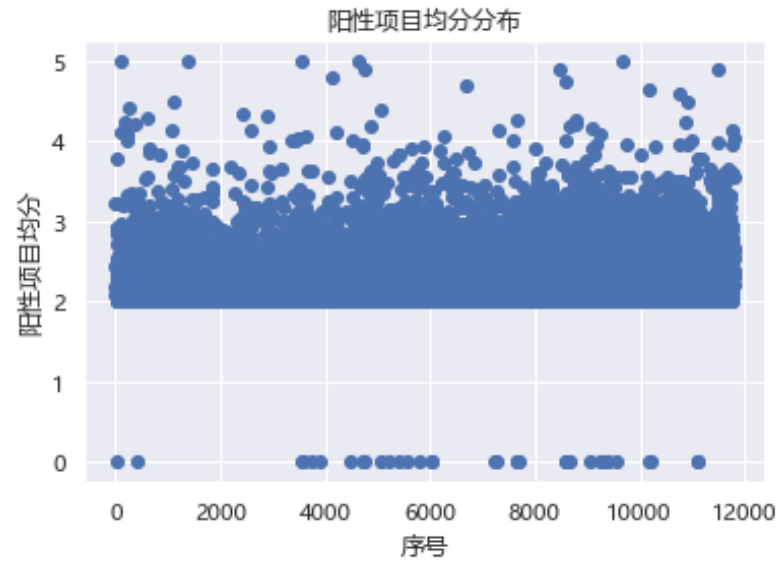


图 5: 阳性项目均分分布

3.6 重要特征之间的散点图

然后我们可以绘制出几个重要特征之间的散点图，如人际关系均分, 抑郁均分, 焦虑均分，查看几个重要特征之间的关系

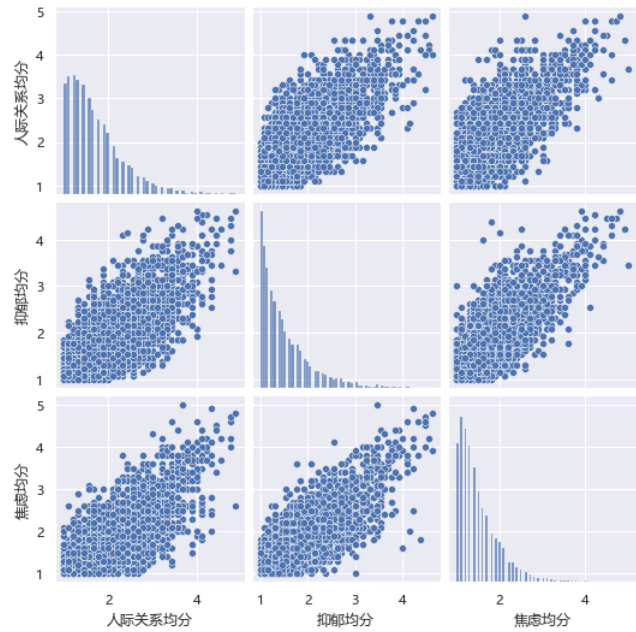


图 6: 重要特征之间的散点图

3.7 各特征相关系数

所下图所示，图中颜色越红表示该点的横纵坐标所代表的特征的相关性越大，例如躯体化均分和总分的相关系数为 0.79；强迫症和总分的相关系数为 0.88，主对角线是自己与自己的相关系数为 1。

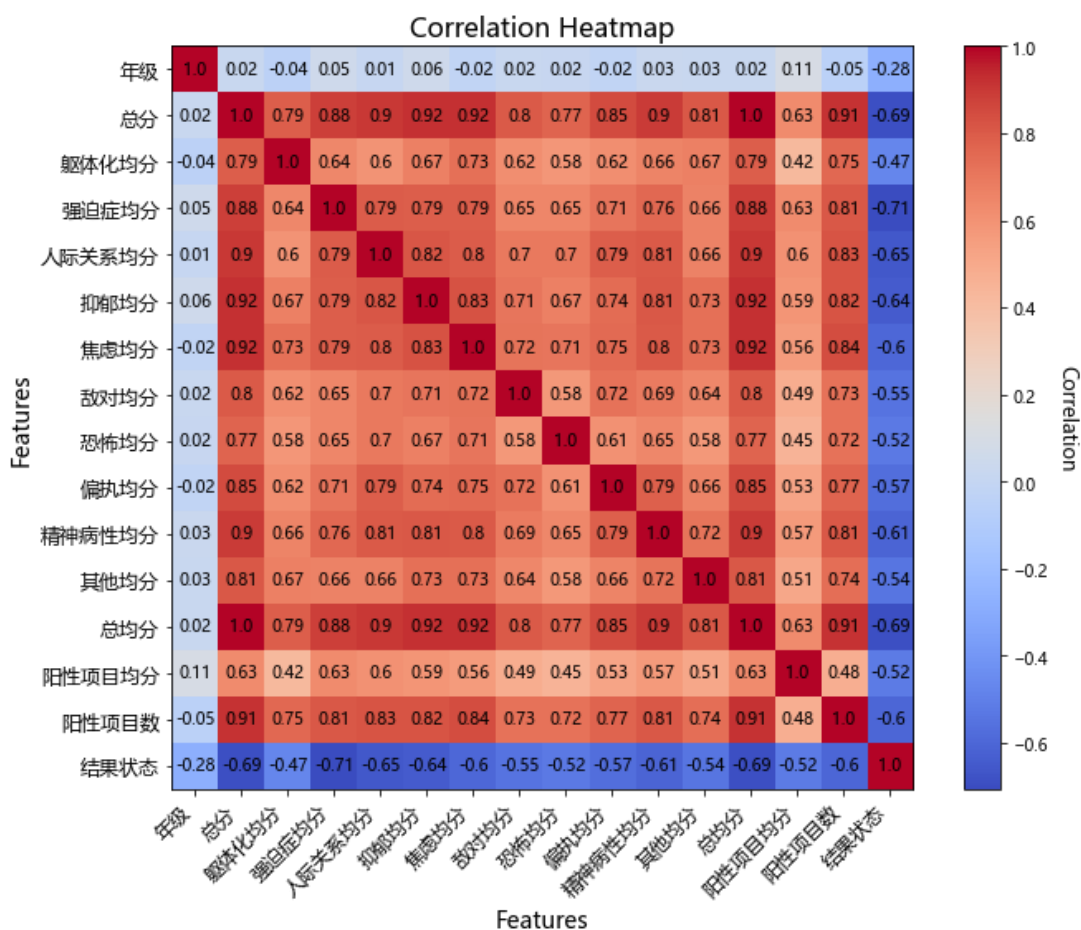


图 7: 各特征相关系数

4 模型和算法简介

4.1 决策树 (Decision Tree)

4.1.1 原理

决策树的原理基于信息熵的概念。信息熵是度量一个随机变量不确定性的度量，其值越大，表示随机变量的不确定性越高。在决策树中，使用信息熵来度量一个特征对分类结果的影响程度，选择对分类结果影响最大的特征作为当前节点的划分特征。

C4.5 算法是用于生成决策树的一种经典算法，是 ID3 算法的一种延伸和优化。C4.5 算法对 ID3 算法进行了改进，本文主要是以 C4.5 算法构建决策数

决策树的构建过程如下：

首先，需要计算数据集中每个特征的信息增益比。信息增益比是信息增益除以特征熵的值，其中信息增益表示使用该特征进行划分所能获得的信息量，特征熵表示使用该特征进行划分时所需要的信息量。信息增益比能够避免选择取值较多的特征作为划分特征，从而使得决策树更加简洁。

然后，选择信息增益比最大的特征作为当前节点的划分特征，并根据该特征的取值将数据集划分为多个子集。

对于每个子集，递归地进行步骤 1 和步骤 2，直到满足终止条件。终止条件可以是节点的深度达到预设的最大深度，或者节点中的样本数达到预设的最小样本数。

最后，得到了一棵决策树，可以用于对新数据进行分类。

4.1.2 关键概念

- **信息熵 (Entropy):** 度量数据不确定性，计算公式为

$$E(S) = - \sum_{i=1}^n p_i \log_2 p_i$$

其中 p_i 是第 i 类样本在数据集中的比例。

- **信息增益 (Information Gain):** 选择特征对数据集划分前后不确定性的减少量，计算公式为

$$IG(S, A) = E(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} E(S_v)$$

其中 S_v 是特征 A 的一个值对应的子集。

- **信息增益比 (Gain Ratio):** 信息增益除以特征的固有信息量，用于减少对多值特征的偏好。

4.1.3 构建过程

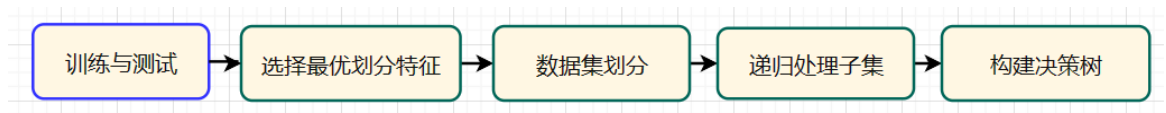


图 8: 决策树的构建过程

4.2 支持向量机 (SVM)

4.2.1 原理

SVM 的原理基于间隔最大化的思想。对于一个二分类问题，假设我们的数据集是 $\{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$ ，其中 x_i 是样本的特征向量， $y_i \in \{+1, -1\}$ 是样本的标签。

我们的目标是找到一个超平面 $wx + b = 0$ ，将正负样本分开。其中 w 是超平面的法向量， b 是偏置项。

我们定义样本到超平面的距离为间隔，用 γ 表示。对于一个样本 (x_i, y_i) ，它到超平面的距离 γ_i 等于 $\frac{y_i(wx+b)}{\|w\|}$ ，其中 $\|w\|$ 表示 w 的模长。对于所有的正样本和负样本，我们都可以计算出它们到超平面的距离 γ_i 。我们的目标是找到一个超平面，使得所有正样本到超平面的距离 γ_+ 和所有负样本到超平面的距离 γ_- 的差值最大。也就是说，我们要最大化间隔 $\gamma_+ - \gamma_-$ 。

由于超平面 $wx + b = 0$ 可以通过调整 w 和 b 来确定，因此我们可以将最大化间隔的问题转化为一个优化问题。具体来说，我们可以将最大化间隔的问题转化为一个凸二次规划问题，用拉格朗日乘子法求解。最终，我们可以得到一个最优的超平面，将正负样本分开。

对于非线性可分的问题，我们可以使用核函数来将数据映射到高维空间中。在高维空间中，我们可以找到一个超平面将数据分开。由于在高维空间中计算复杂度较高，因此我们可以使用核函数来避免显式地计算高维空间中的向量。常用的核函数有线性核函数、多项式核函数、高斯核函数等。

4.2.2 关键概念

- 超平面 (Hyperplane): $wx + b = 0$ 形式，其中 w 是权重， b 是偏置。
- 间隔 (Margin): 数据点到超平面的最短距离。
- 拉格朗日乘子法 (Lagrange Multipliers): 用于求解约束优化问题。

4.2.3 优化问题

最大化间隔的优化问题表述为：

1. 最小化 $\frac{1}{2}\|w\|^2$ 。
2. 约束条件：对每个样本 (x_i, y_i) ，满足 $y_i(wx_i + b) \geq 1$ 。

4.2.4 核函数 (Kernel Function)

用于处理非线性可分问题，通过将数据映射到高维空间使其线性可分。

4.3 随机森林 (Random Forest)

4.3.1 原理

随机森林的原理基于 Bagging 和随机化的思想。Bagging 是一种基于自助采样的集成学习算法，它通过从原始数据集中有放回地随机采样，生成多个数据集，并使用这些数据

集训练多个分类器或回归器。随机化是一种随机选择特征的方法，它可以减少决策树的过拟合。

对于一个二分类或回归问题，假设我们的数据集是 $\{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$ ，其中 x_i 是样本的特征向量， y_i 是样本的标签。我们使用 **Bagging** 的思想，从原始数据集中有放回地随机采样，生成 m 个数据集。对于每个数据集，我们使用随机化的方法，从原始特征中随机选择 k 个特征，构建一棵决策树。在构建决策树时，采用 **CART** 算法，选择最佳的特征进行分裂，并递归地构建子树。最终，我们得到了 m 棵决策树。

在进行分类或回归时，使用投票或平均的方法，将每个决策树的结果进行集成。对于分类问题，选择出现最多的类别作为最终的分类结果；对于回归问题，选择所有决策树的输出的平均值作为最终的回归结果。

随机森林的优点是能够处理高维数据和非线性可分问题，具有很好的泛化能力和较高的分类精度。同时，随机森林对于噪声数据比较鲁棒，不容易过拟合。但是随机森林也存在一些缺点，比如对于数据分布不平衡的问题，需要进行调整。

4.3.2 构建过程

1. 自助采样 (Bootstrap Sampling): 从原始数据集中有放回地随机抽样，形成多个子集。
2. 构建决策树: 对每个子集，随机选择特征子集，基于这些特征构建决策树。
3. 集成:
 - 分类任务: 通过多数投票确定最终分类。
 - 回归任务: 取多个决策树预测结果的平均值。

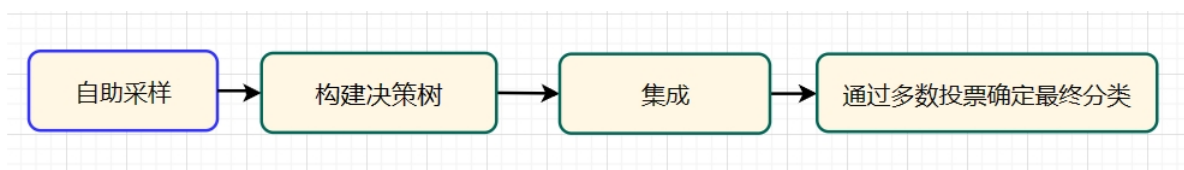


图 9: 随机森林的构建过程

4.4 混淆矩阵 (Confusion Matrix)

4.4.1 定义

混淆矩阵是用于评估分类模型性能的一种常见方法。混淆矩阵将真实标签和预测标签进行比较，从而得出模型的分类准确率、召回率、精度和 **F1** 分数等指标。混淆矩阵可以帮助我们了解分类模型的性能和误差类型，从而进一步优化模型。

混淆矩阵是一个二维矩阵，行表示真实标签，列表示预测标签。在二分类问题中，混淆矩阵通常由四个元素组成：真正例 (True Positive, TP)、假正例 (False Positive, FP)、

真反例 (True Negative, TN) 和假反例 (False Negative, FN)。其中, 真正例表示模型正确地将正例分类为正例的数量, 假正例表示模型错误地将负例分类为正例的数量, 真反例表示模型正确地将负例分类为负例的数量, 假反例表示模型错误地将正例分类为负例的数量。

根据混淆矩阵的元素, 我们可以计算出多个分类指标。准确率表示模型正确分类的样本数占总样本数的比例, 计算公式为 $\frac{TP+TN}{TP+FP+TN+FN}$ 。召回率 (Recall) 表示正例被正确分类的比例, 计算公式为 $\frac{TP}{TP+FN}$ 。精度 (Precision) 表示被分类为正例的样本中真正为正例的比例, 计算公式为 $\frac{TP}{TP+FP}$ 。F1 分数 (F1-Score) 是准确率和召回率的加权平均值, 计算公式为 $\frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$ 。

4.4.2 组成

- 真正例 (True Positive, TP): 正确地将正例预测为正例。
- 假正例 (False Positive, FP): 错误地将负例预测为正例。
- 真负例 (True Negative, TN): 正确地将负例预测为负例。
- 假负例 (False Negative, FN): 错误地将正例预测为负例。

4.4.3 主要指标

- 准确率 (Accuracy): $\frac{TP+TN}{TP+FP+TN+FN}$
- 召回率 (Recall): $\frac{TP}{TP+FN}$
- 精确率 (Precision): $\frac{TP}{TP+FP}$
- F1 分数 (F1 Score): $2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$

4.5 ROC 曲线 (Receiver Operating Characteristic Curve)

4.5.1 定义

ROC 曲线是通过以真正例率为纵轴, 假正例率为横轴, 绘制而成的一条曲线, 用于评估二分类模型的性能。真正例率表示模型将正例正确分类为正例的比例, 计算公式为 $\frac{TP}{TP+FN}$; 假正例率表示模型错误地将负例分类为正例的比例, 计算公式为 $\frac{FP}{FP+TN}$ 。通过在不同阈值下计算模型在不同的真正例率和假正例率之间的权衡, 可以绘制出 ROC 曲线。

在 ROC 曲线上, 左下角是 (0,0) 点, 表示将所有样本都预测为负例; 右上角是 (1,1) 点, 表示将所有样本都预测为正例。曲线离左上角越近, 模型的性能越好。曲线下方的面积被称为 AUC (Area Under Curve), AUC 越大, 代表模型的性能越好。

4.5.2 关键概念

- 真正例率 (True Positive Rate, TPR): $\frac{TP}{TP+FN}$
- 假正例率 (False Positive Rate, FPR): $\frac{FP}{FP+TN}$

4.5.3 AUC (Area Under Curve)

AUC 衡量了 ROC 曲线下的面积, AUC 越大, 模型性能通常越好。

5 模型结果分析

5.1 决策树

表 4: 决策树 classification performance

	precision	recall	f1-score
	0.99	0.99	0.99
accuracy			0.99
macro avg	0.99	0.99	0.99
weighted avg	0.99	0.99	0.99

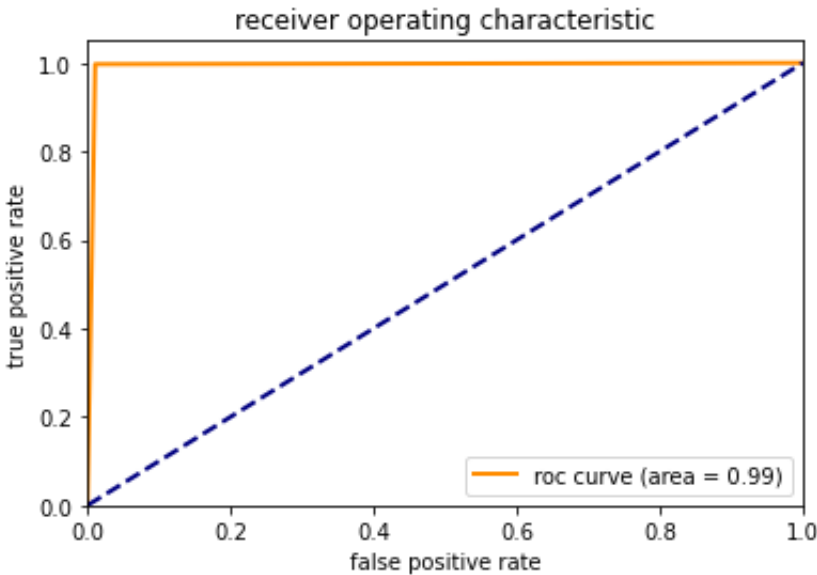


图 10: 决策树

- 精确度 (Precision), 召回率 (Recall), F1 分数 (F1-Score): 这些指标对于决策树模型均为 0.99, 指出其在正负类别预测上的准确性非常高。

- AUC 值：AUC 值为 0.99，接近 1，暗示模型在区分类别上的表现极佳。
- 分析：决策树模型在所有评价指标上均表现出色。高精确度意味着较低的假阳性率，高召回率则表明它几乎不会错过任何真实的正例。F1 分数确认了模型在精确度和召回率上的平衡性能，而高 AUC 值则表示该模型在区分正负类别上有着强大的能力。

5.2 SVM

表 5: SVM classification performance

	precision	recall	f1-score
	0.90	0.88	0.89
accuracy			0.94
macro avg	0.93	0.92	0.93
weighted avg	0.94	0.94	0.94

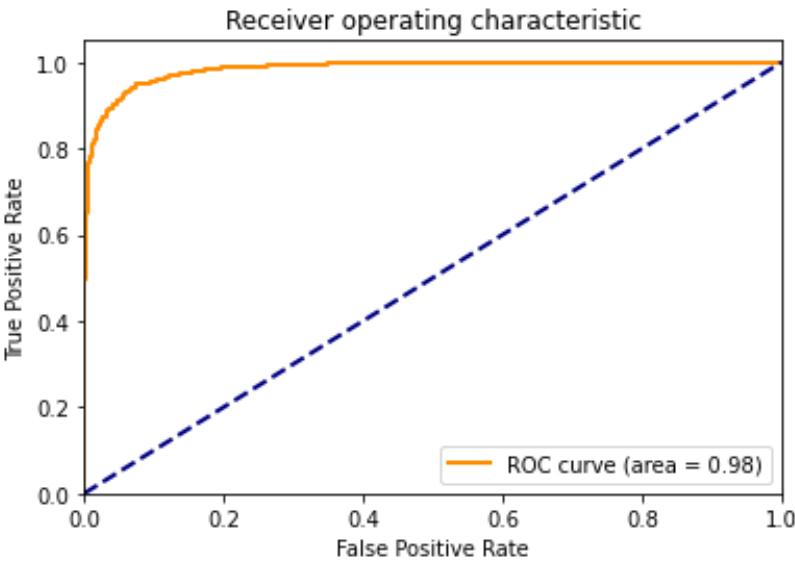


图 11: SVM

- 精确度，召回率，F1 分数：SVM 的精确度为 0.90，召回率为 0.88，F1 分数为 0.89。虽然这些是不错的得分，但没有达到 C4.5 决策树的高度。
- AUC 值：SVM 的 AUC 值为 0.98，稍低于决策树，但仍然显示出良好的预测能力。
- 分析：从上面两个 ROC 曲线可以看出，C4.5 决策树的 AUC 值为 0.99，而 SVM 的 AUC 值为 0.98。然而，C4.5 决策树在 precision、recall、f1-score 指标上的得分都为 0.99，而 SVM 的 precision 为 0.9，recall 为 0.88，f1-score 为 0.89。这表明在这个问题上，C4.5 决策树的性能要比 SVM 模型更为出色，具有更高的准确度。

5.3 随机森林

表 6: 随机森林 classification performance

	precision	recall	f1-score
	0.99	0.97	0.98
accuracy			0.99
macro avg	0.99	0.98	0.98
weighted avg	0.99	0.99	0.99

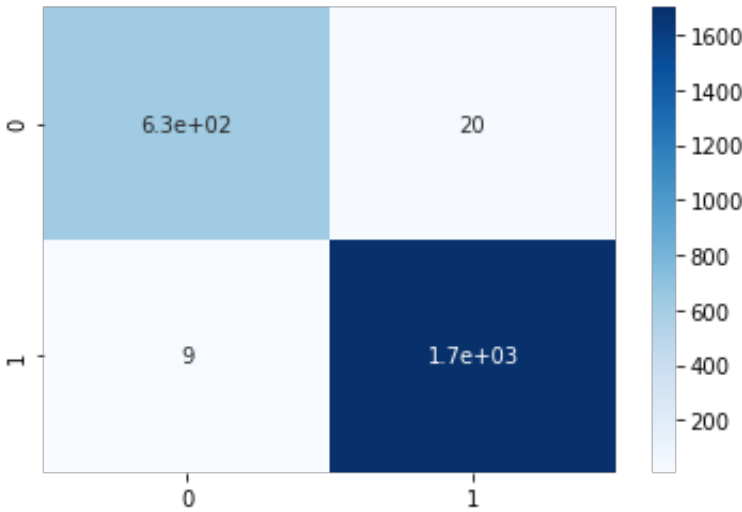


图 12: 随机森林

- 精确度，召回率，F1 分数：随机森林的精确度为 0.99，召回率为 0.97，F1 分数为 0.98。
- 准确度：准确度同样达到了 0.99，与高精确度和召回率相匹配。
- 混淆矩阵：模型正确预测了 630 个类别 0 的实例和 1700 个类别 1 的实例，仅有 9 个假阴性和 20 个假阳性。
- 分析：随机森林模型展现了卓越的结果，高精确度意味着假阳性很少，而高召回率则意味着它捕获了大部分正例。F1 分数验证了该模型在平衡精确度和召回率上的优良性能，混淆矩阵进一步证实了模型的高准确率和强大的预测力，仅有极少的误判。

其混淆矩阵如图所示，有 630 个原本为 0 的它预测还是 0，有 1700 个原本为 1 的他预测还是 1，有 9 个原本是 1 的他预测成了 0 没有 20 个原本是 0 的它预测成了 1，说明随机森林的结果还是不错的。

5.4 综合比较

- 准确度：所有模型均显示出高度的准确率，其中随机森林和决策树略微优于 SVM。
- 稳健性：从 F1 分数和 AUC 值来看，决策树和随机森林模型比 SVM 更为稳健。
- 模型选择：如果必须基于提供的指标选择模型，决策树似乎是最为精确的。然而，随机森林同样展示了类似的性能，并可能由于其集成的特性而提供更好的泛化能力。

5.5 考虑因素

- 模型复杂性：决策树可能存在过拟合的问题，特别是在得分完美的情况下。重要的是，需要使用独立的测试集来验证这些结果的有效性。
- 数据平衡：所有模型的性能指标都非常高，这也可能表明问题本身较为简单，或者数据并不是高度不平衡的。
- 用例：根据应用场景的不同，少量的假阳性和假阴性可能带来的影响也不同。例如，在医疗诊断中，一个假阴性通常比假阳性的代价要高。

6 总结与评价

6.1 总结

大学生心理素质整体上表现良好，但仍存在一定比例的学生存在心理问题，需要关注和帮助。

1. 决策树：在测试集上几乎达到了完美的分类效果，所有评价指标均为 0.99，显示出了在当前问题上的强大性能。这表明该模型能够非常准确地对正负样本进行分类。在心理素质分析中表现出色，可作为心理素质分析的首选模型。
2. SVM（支持向量机）：SVM 表现出了良好的分类能力，但是在精确度、召回率和 F1 分数方面稍逊于决策树。尽管如此，其 AUC 值仍然很高，表明它在大多数情况下能够可靠地区分不同类别。虽然比不上前两个模型，但仍有一定的应用价值。
3. 随机森林：随机森林的性能与决策树非常接近，表明其对于分类问题同样有着出色的解决能力。它在处理大规模数据集时通常更加稳健，这得益于它的集成学习特性。表现也非常优秀，可以作为决策树模型的备选模型。

6.2 评价

- 决策树的强大性能：需要警惕过拟合的可能性，尤其是在数据集较小或特征数目较多时。但如果模型确实准确，那么它在实际应用中可能非常有价值。
- SVM 的实用性：SVM 的性能可能在特征空间复杂或数据分布复杂的问题上更为突出，且对异常值具有较强的鲁棒性。
- 随机森林的泛化能力：通常具有较好的泛化能力和抗噪声能力，适用于各种数据类型和问题。

在选择模型时，除了性能指标外，还应考虑模型的可解释性、计算成本、以及在实际应用中的可维护性。例如，虽然随机森林在解释模型决策过程方面可能不如决策树，但它通常提供更好的预测准确性和抗噪声能力。

在实践中，还应通过交叉验证和其他模型验证技术来评估模型的实际性能，以确保选定的模型既能在当前数据集上表现良好，也能对未知数据进行有效的泛化。验证这些模型在现实世界场景中或使用未见过的数据以确保它们没有过拟合且能够很好地泛化是至关重要的。

综上所述，每种模型都有其优势和局限性，应根据具体的应用场景和需求来选择最合适的模型。本文中研究发现，对于大一新生心理测试数据集，采用决策树模型最好，能够非常准确地对正负样本进行分类。在心理素质分析中表现出色，可作为心理素质分析的最优首选模型。

7 模型改进建议

- 对于决策树和随机森林，建议进行交叉验证以及不同参数的网格搜索，以确保模型的泛化能力。
- SVM 模型可以尝试更多的核函数，并进行细致的参数优化，以期望获得更佳的分类型效果。
- 引入成本敏感学习或重采样技术来处理潜在的数据不平衡问题，以提高模型在各类别上的分类性能。
- 探索自适应方法和在线学习策略，以使模型能够随着新数据的到来而更新和适应。

这些内容不仅提供了对当前模型性能的深度分析，而且指明了未来研究的方向和优化模型的具体途径。通过不断的迭代和改进，我们可以期待模型在未来的表现会更加出色。

参考文献

- [1] 李金鹏. 艺术类高校大学生心理健康教育研究. 才智, 2023(17):96–99.
- [2] 钟晓虹, 石新娣. 大学生心理健康教育要义及规训路径研究——基于三维整合型视域. 岳阳职业技术学院学报, 2021(06):77–80.
- [3] 秦娟. 职业院校“心理育人”理念探析. 四川劳动保障, 2023(05):63–64.
- [4] 岳扬, 山成虎, 彭春霖, 郑小琳, 闫玉玺, 谷琼. 基于新浪微博数据的东航 MU5735 坠毁事件舆情热度演化及情感分析. 计算机时代, 2023,(02):55–60.
- [5] 完颜兵, 张超群, 王大睿, 李晓翔, 郝小芳. 基于网评数据的游客印象挖掘与情感分析. 情报工程, 2023, 9(01):15–29.
- [6] 彭梅, 胡必波. 基于大数据人工智能的电商用户评论情感分析. 电脑编程技巧与维护, 2022,(06):123–126.
- [7] 张黎娜, 张月蓉, 童敏. 基于成人学习者在线文本数据的挖掘和情感分析. 安徽开放大学学报, 2022,(02):93–96.
- [8] 房立艳, 张大均, 刘广增, 李淑芬, 陈合武. 心理素质对中学生拖延的影响: 潜在类别分析. 西南大学学报 (自然科学版), 2021, 43(07):30–35.
- [9] 金琳, 张大均, 刘广增. 父母教养方式与班级氛围对中学生心理素质的影响: 基于多层线性分析. 心理研究, 2021,14(03):273–280.
- [10] 陈小红, 马春雷, 邓砚, 杨鑫. 我国大学生心理健康研究现状的可视化图谱分析. 职业与健康, 2021,37(11):1544–1547.

课程感言与建议

这一学期学习《数据挖掘》这门课，感谢陈灵老师对我的培养，学生朽木，老师常用生活化的例子将课变得浅显易懂，并且课上会让我们自己运行程序并尝试解决其中的报错，加深了我们解决问题和实操能力，十分感谢老师这一学期的付出。每次跟老师一起讨论都有新的心得体会，受益颇多。建议可以再多一点项目式的作业，从项目与实操上学习模型可能对原理上会有更多的体会。

附 录

附录 1：预处理 + 特征工程 + 可视化

```
1 #!/usr/bin/env python
2 # coding: utf-8
3
4 # # 数据预处理
5 # 导入所需库
6 import pandas as pd
7 import numpy as np
8 import matplotlib.pyplot as plt
9
10 # 读取训练数据集.xlsx文件
11 df = pd.read_excel('训练集.xlsx')
12 df
13
14 # 检查缺失值
15 print("缺失值数量：\n", df.isnull().sum())
16
17 # 描述数据分布
18 print("数据分布描述统计信息：\n", df.describe())
19
20 # 检查重复值
21 print("重复值数量：\n", df.duplicated().sum())
22
23 # 抛出重复值
24 df.drop_duplicates(inplace=True)
25 df
26 # # 特征工程+可视化
27
28 import pandas as pd
29 from sklearn.ensemble import RandomForestClassifier
30
31 # 提取特征列和标签列
32 X = df.drop(['性别', '学院', '结果状态'], axis=1) # 特征列（不包括性别和学院）
33 y = df['结果状态'] # 标签列
34
35 # 训练随机森林分类器
36 rfc = RandomForestClassifier(n_estimators=500, random_state=42)
```

```
37 rfc . fit (X, y)
38
39 # 输出特征重要性得分，找出前5个得分最高的特征
40 importances = rfc . feature_importances_
41 indices = (-importances). argsort () [:5]
42 print ('5个最重要的特征：')
43 for i in indices :
44     print (X.columns[i])
45
46 # 统计年级列数量
47 grade_counts = df['年级']. value_counts ()
48 grade_counts
49
50 import matplotlib . font_manager as fm
51 import seaborn as sns
52 import matplotlib . pyplot as plt
53
54 # 设置中文字体
55 font_name = fm. FontProperties (fname='C:/Windows/Fonts/msyh.ttc'). get_name()
56 plt . rcParams[' font . family'] = font_name
57
58 # 绘制柱状图
59 plt . figure ( figsize =(8,6))
60 sns . barplot (x=grade_counts.index, y=grade_counts.values, palette='pastel')
61
62 # 添加标签
63 plt . title ('各年级人数分布', fontsize=14)
64 plt . xlabel ('年 级', fontsize=12)
65 plt . ylabel ('人 数', fontsize=12)
66
67 # 修改字体和样式
68 plt . xticks ( fontsize=10)
69 plt . yticks ( fontsize=10)
70 plt . tight_layout ()
71
72 # 显示图像
73 plt . show()
74
75 # 统计性别列数量
```

```
76 sex_counts = df['性别'].value_counts()
77 sex_counts
78
79 import matplotlib.pyplot as plt
80
81 # 数据
82 sex_counts = {"男性": 3251, "女性": 8131}
83
84 # 设置中文字体
85 font_name = fm.FontProperties(fname='C:/Windows/Fonts/msyh.ttc').get_name()
86 plt.rcParams['font.family'] = font_name
87
88 # 绘制饼图
89 plt.figure(figsize=(6, 6))
90 colors = ['#ff9999', '#66b3ff']
91 plt.pie(x=sex_counts.values(), labels=sex_counts.keys(), autopct='%0.1f%%', colors=
    colors)
92
93 # 添加标题和图例
94 plt.title("性别分布")
95 plt.legend(loc='best')
96
97 # 显示图像
98 plt.show()
99
100 import pandas as pd
101 import matplotlib.pyplot as plt
102
103 # 读取数据框df并计算学院数量
104 college_counts = df['学院'].value_counts().head(10)
105
106 # 绘制横向条形图
107 fig, ax = plt.subplots(figsize=(8, 6))
108 ax.barh(college_counts.index, college_counts.values)
109
110 # 添加标题和标签
111 ax.set_title('排名前10的学院', fontsize=16)
112 ax.set_xlabel('数量', fontsize=14)
113 ax.set_ylabel('学院', fontsize=14)
```

```
114
115 # 显示图像
116 plt.show()
117
118 import pandas as pd
119 import matplotlib.pyplot as plt
120
121 # 设置中文字体
122 font_name = fm.FontProperties(fname='C:/Windows/Fonts/msyh.ttc').get_name()
123 plt.rcParams['font.family'] = font_name
124
125 # 统计每个强迫症等级的人数
126 normal_count = ((df['强迫症均分'] >= 0) & (df['强迫症均分'] < 1.5)).sum()
127 mild_count = ((df['强迫症均分'] >= 1.5) & (df['强迫症均分'] < 2.5)).sum()
128 severe_count = (df['强迫症均分'] >= 2.5).sum()
129 print(normal_count)
130 print(mild_count)
131 print(severe_count)
132 # 绘制柱状图
133 counts = [normal_count, mild_count, severe_count]
134 labels = ['正常', '轻度强迫症', '重度强迫症']
135 plt.bar(labels, counts)
136
137 # 添加图表标题和轴标签
138 plt.title('强迫症等级分布')
139 plt.xlabel('等级')
140 plt.ylabel('人数')
141
142 # 显示图表
143 plt.show()
144
145 import pandas as pd
146 import matplotlib.pyplot as plt
147
148 # 绘制散点图
149 plt.scatter(x=df.index, y=df['阳性项目均分'])
150
151 # 添加图表标题和轴标签
152 plt.title('阳性项目均分分布')
```

```
153 plt.xlabel('序号')
154 plt.ylabel('阳性项目均分')
155
156 # 显示图表
157 plt.show()
158
159 import pandas as pd
160 import matplotlib.pyplot as plt
161
162 # 将总分列离散化
163 bins = [0, 100, 200, 300, 400]
164 df['总分范围'] = pd.cut(df['总分'], bins=bins)
165 counts = df['总分范围'].value_counts()
166
167 # 绘制柱状图
168 labels = ['0-100', '100-200', '200-300', '300-400']
169 plt.bar(labels, counts)
170
171 # 添加图表标题和轴标签
172 plt.title('总分分布')
173 plt.xlabel('总分范围')
174 plt.ylabel('人数')
175
176 # 显示图表
177 plt.show()
178
179 import pandas as pd
180 import matplotlib.pyplot as plt
181
182 # 将抑郁均分列离散化
183 bins = [0, 1.5, 2.5, 10]
184 df['抑郁等级'] = pd.cut(df['抑郁均分'], bins=bins, labels=['正常', '轻度抑郁', '抑郁'])
185
186 # 统计结果状态为0的数量
187 count = ((df['抑郁等级'] == '正常') & (df['结果状态'] == 0)).sum()
188
189 # 绘制饼图
190 labels = ['正常', '其他']
```

```
191 sizes = [count, len(df) - count]
192 explode = (0.1, 0)
193 plt.pie(sizes, explode=explode, labels=labels, autopct='%1.1f%%', shadow=True,
194         startangle=90)
195
196 # 添加图表标题
197 plt.title('心理检测不健康的人数中不抑郁的人数占比')
198
199 # 显示图表
200 plt.show()
201
202 import pandas as pd
203 import matplotlib.pyplot as plt
204
205 # 将焦虑均分列离散化
206 bins = [0, 1.5, 2.5, 10]
207 df['焦虑等级'] = pd.cut(df['焦虑均分'], bins=bins, labels=['正常', '轻度焦虑', '焦
208     虑'])
209
210 # 统计结果状态为0的数量
211 count = ((df['焦虑等级'] == '正常') & (df['结果状态'] == 0)).sum()
212
213 # 绘制饼图
214 labels = ['正常', '其他']
215 sizes = [count, len(df) - count]
216 explode = (0.1, 0)
217 plt.pie(sizes, explode=explode, labels=labels, autopct='%1.1f%%', shadow=True,
218         startangle=90)
219
220 # 添加图表标题
221 plt.title('心理检测不健康的人数中不焦虑人数占比')
222
223 # 显示图表
224 plt.show()
225
226 import pandas as pd
227 import seaborn as sns
```

```
227
228 # 筛选需要的特征列
229 features = ['人际关系均分','抑郁均分','焦虑均分']
230 df = df[ features ]
231
232 # 绘制散点图矩阵
233 sns.pairplot(df)
234
235 # 显示图表
236 plt.show()
237
238 import pandas as pd
239 import seaborn as sns
240
241 # 绘制各特征之间的散点图
242 sns.pairplot(df, hue='结果状态')
243
244 # 显示图表
245 plt.show()
246
247 # 导入库
248 import pandas as pd
249 import matplotlib.pyplot as plt
250 import matplotlib.font_manager as fm
251 # 设置中文字体
252 font_name = fm.FontProperties(fname='C:/Windows/Fonts/msyh.ttc').get_name()
253 plt.rcParams['font.family'] = font_name
254 # 读取数据集
255 data = pd.read_excel('训练集.xlsx')
256
257 # 获取特征之间的相关系数矩阵
258 corr_matrix = data.corr()
259
260 # 绘制相关系数矩阵的热力图
261 fig, ax = plt.subplots(figsize=(10,8))
262 im = ax.imshow(corr_matrix, cmap='coolwarm')
263
264 # 显示相关系数数值
265 for i in range(len(corr_matrix.columns)):
```

```

266     for j in range(len(corr_matrix.columns)):
267         text = ax.text(j, i, round(corr_matrix.iloc[i,j],2),
268                             ha='center', va='center', color='black')
269
270 # 设置图像参数
271 ax.set_xticks(range(len(corr_matrix.columns)))
272 ax.set_yticks(range(len(corr_matrix.columns)))
273 ax.set_xticklabels(corr_matrix.columns, rotation=45, ha='right', fontsize=12)
274 ax.set_yticklabels(corr_matrix.columns, fontsize=12)
275 ax.set_title('Correlation Heatmap', fontsize=16)
276 ax.set_xlabel('Features', fontsize=14)
277 ax.set_ylabel('Features', fontsize=14)
278
279 # 添加颜色条
280 cbar = ax.figure.colorbar(im, ax=ax)
281 cbar.ax.set_ylabel('Correlation', rotation=-90, va='bottom', fontsize=12)
282
283 # 显示图像
284 plt.tight_layout()
285 plt.show()

```

附录 2：模型

```

1  #!/usr/bin/env python
2  # coding: utf-8
3
4  ## 随机森林+混淆矩阵
5
6  import pandas as pd
7  from sklearn.ensemble import RandomForestClassifier
8  from sklearn.model_selection import train_test_split
9  from sklearn.metrics import classification_report, confusion_matrix
10 import seaborn as sns
11
12 # 读取训练集
13 train_data = pd.read_excel('训练集.xlsx')
14
15 # 读取测试集
16 test_data = pd.read_excel('测试集.xlsx')
17

```

```
18 # 筛选特征列和标签列
19 features = ['年级', '总分', '躯体化均分', '强迫症均分', '人际关系均分', '抑郁均分',
              '焦虑均分', '敌对均分', '恐怖均分', '偏执均分', '精神病性均分', '其他均分', '总
              均分', '阳性项目均分', '阳性项目数']
20 label = '结果状态'
21
22 # 分离训练集特征和标签
23 X_train = train_data [ features ]
24 y_train = train_data [ label ]
25
26 # 分离测试集特征和标签
27 X_test = test_data [ features ]
28
29 # 将训练集数据进一步拆分为训练集和验证集
30 X_train, X_val, y_train, y_val = train_test_split (X_train, y_train, test_size =0.2,
              random_state=42)
31
32 # 实例化随机森林分类器模型
33 rfc = RandomForestClassifier( n_estimators=100)
34
35 # 在训练集上训练模型
36 rfc . fit (X_train, y_train )
37
38 # 在验证集上测试模型预测效果
39 y_pred = rfc . predict (X_val)
40
41 # 输出分类报告和混淆矩阵
42 print ( classification_report (y_val, y_pred))
43 cm = confusion_matrix(y_val, y_pred)
44 sns.heatmap(cm, annot=True, cmap='Blues')
45
46 # 在测试集上测试模型预测效果
47 y_test_pred = rfc . predict (X_test)
48
49 # 输出测试集预测结果
50 print ( y_test_pred )
51
52 #在测试集上测试模型预测效果
53 y_test_pred = rfc . predict (X_test)
```

```
54
55 #将预测结果作为新列添加到测试数据集
56 test_data ['预测结果'] = y_test_pred
57
58 #将预测结果为0的改为“预警”
59 test_data .loc[ test_data ['预测结果'] == 0, '预测结果'] = '预警'
60
61 #将带有预测结果的测试数据集保存为一个新的数据集
62 test_data .to_excel('随机森林预测结果数据集.xlsx', index=False)
63
64 # # SVM+ROC曲线
65
66 import pandas as pd
67 from sklearn.svm import SVC
68 from sklearn.model_selection import train_test_split
69 from sklearn.metrics import classification_report , confusion_matrix, roc_curve, auc
70 import matplotlib.pyplot as plt
71
72 # 读取训练集和测试集
73 train_data = pd.read_excel('训练集.xlsx')
74 test_data = pd.read_excel('测试集.xlsx')
75
76 # 筛选特征列和标签列
77 features = ['年级', '总分', '躯体化均分', '强迫症均分', '人际关系均分', '抑郁均分',
78             '焦虑均分', '敌对均分', '恐怖均分', '偏执均分', '精神病性均分', '其他均分', '总
79             均分', '阳性项目均分', '阳性项目数']
80 label = '结果状态'
81
82 # 分离训练集特征和标签
83 x_train = train_data [ features ]
84 y_train = train_data [ label ]
85
86 # 分离测试集特征和标签
87 x_test = test_data [ features ]
88
89 # 将训练集数据进一步拆分为训练集和验证集
90 x_train, x_val, y_train, y_val = train_test_split (x_train, y_train, test_size=0.2,
91                                                    random_state=42)
```

```
90 # 实例化SVM模型
91 svm = SVC(kernel='linear', probability=True)
92
93 # 在训练集上训练模型
94 svm.fit(x_train, y_train)
95
96 # 在验证集上测试模型预测效果
97 y_pred = svm.predict(x_val)
98
99 # 输出分类报告
100 print(classification_report(y_val, y_pred))
101
102 # 根据验证集上的预测概率绘制ROC曲线
103 y_pred_proba = svm.predict_proba(x_val)[:,1]
104 fpr, tpr, thresholds = roc_curve(y_val, y_pred_proba)
105 roc_auc = auc(fpr, tpr)
106
107 plt.plot(fpr, tpr, color='darkorange', lw=2, label='ROC curve (area = %0.2f)' %
          roc_auc)
108 plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
109 plt.xlim([0.0, 1.0])
110 plt.ylim([0.0, 1.05])
111 plt.xlabel('False Positive Rate')
112 plt.ylabel('True Positive Rate')
113 plt.title('Receiver operating characteristic')
114 plt.legend(loc='lower right')
115 plt.show()
116
117 # 在测试集上测试模型预测效果
118 y_test_pred = svm.predict(x_test)
119
120 # 将预测结果作为新列添加到测试数据集
121 test_data['预测结果'] = y_test_pred
122
123 # 将预测结果为0的改为“预警”
124 test_data.loc[test_data['预测结果'] == 0, '预测结果'] = '预警'
125
126 # 将带有预测结果的测试数据集保存为一个新的数据集
127 test_data.to_excel('SVM预测结果数据集.xlsx', index=False)
```

```
128 ## 决策树
129
130 import pandas as pd
131 from sklearn . tree import DecisionTreeClassifier
132 from sklearn . model_selection import train_test_split
133 from sklearn . metrics import classification_report , confusion_matrix , roc_curve , auc
134 import matplotlib . pyplot as plt
135
136 # 读取训练集和测试集
137 train_data = pd.read_excel('训练集.xlsx')
138 test_data = pd.read_excel('测试集.xlsx')
139
140 # 筛选特征列和标签列
141 features = ['年级', '总分', '躯体化均分', '强迫症均分', '人际关系均分', '抑郁均分',
            '焦虑均分', '敌对均分', '恐怖均分', '偏执均分', '精神病性均分', '其他均分', '总
            均分', '阳性项目均分', '阳性项目数']
142 label = '结果状态'
143
144 # 分离训练集特征和标签
145 x_train = train_data [ features ]
146 y_train = train_data [ label ]
147
148 # 分离测试集特征和标签
149 x_test = test_data [ features ]
150
151 # 将训练集数据进一步拆分为训练集和验证集
152 x_train , x_val , y_train , y_val = train_test_split ( x_train , y_train , test_size =0.2,
            random_state=42)
153
154 # 实例化C4.5决策树模型
155 dt = DecisionTreeClassifier ( criterion ='entropy' , random_state=42)
156
157 # 在训练集上训练模型
158 dt . fit ( x_train , y_train )
159
160 # 在验证集上测试模型预测效果
161 y_pred = dt . predict ( x_val )
162
163 # 输出分类报告
```

```
164 print ( classification_report (y_val, y_pred))
165
166 # 根据验证集上的预测概率绘制roc曲线
167 y_pred_proba = dt.predict_proba(x_val)[:,1]
168 fpr, tpr, thresholds = roc_curve(y_val, y_pred_proba)
169 roc_auc = auc(fpr, tpr)
170
171 plt.plot(fpr, tpr, color='darkorange', lw=2, label='roc_curve(area=%0.2f)' %
          roc_auc)
172 plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
173 plt.xlim([0.0, 1.0])
174 plt.ylim([0.0, 1.05])
175 plt.xlabel('false positive rate')
176 plt.ylabel('true positive rate')
177 plt.title('receiver operating characteristic ')
178 plt.legend(loc="lower right")
179 plt.show()
180
181 # 在测试集上测试模型预测效果
182 y_test_pred = dt.predict(x_test)
183
184 # 将预测结果作为新列添加到测试数据集
185 test_data['预测结果'] = y_test_pred
186
187 # 将预测结果为0的改为“预警”
188 test_data.loc[test_data['预测结果'] == 0, '预测结果'] = '预警'
189
190 # 将带有预测结果的测试数据集保存为一个新的数据集
191 test_data.to_excel('决策树预测结果数据集.xlsx', index=False)
```